

CSR-RAG: An Efficient Retrieval System for Text-to-SQL on the Enterprise Scale

Anonymous ACL submission

Abstract

Natural language to SQL translation (*Text-to-SQL*) is one of the long-standing problems that has recently benefited from advances in Large Language Models (LLMs). While most academic *Text-to-SQL* benchmarks request schema description as a part of natural language input, enterprise-scale applications often require table retrieval before SQL query generation. To address this need, we propose a novel hybrid Retrieval Augmented Generation (RAG) system consisting of contextual, structural, and relational retrieval (*CSR-RAG*) to achieve computationally efficient yet sufficiently accurate retrieval for enterprise-scale databases. Through extensive enterprise benchmarks, we demonstrate that *CSR-RAG* achieves up to 40% precision and over 80% recall while incurring a negligible average query generation latency of only 30ms on commodity data center hardware, which makes it appropriate for modern LLM-based enterprise-scale systems.

1 Introduction

«Novak: The problem of baseline. Reviewer 3 is one to point that out.» «Novak: Reviewer 3: do Google Scholar search for "value detection" in NL to SQL. Why just selecting the right tables is justifiable as a solution. —> state of the art works only with BIRD and SPIDER.» «Rajpreet: This is an essential step fundamentally. lets discuss in detail »

Natural language-processing neural networks (NN) have recently shown significant advancements in generative tasks (Radford et al., 2018; Guo et al., 2025), building upon the capabilities of the *transformer* neural network architecture and the versatility of attention mechanisms (Vaswani et al., 2017). Since their introduction, transformer-based NNs have continually improved their accuracy on tasks that involve large, well-defined evaluation sets, including a class of tasks commonly referred

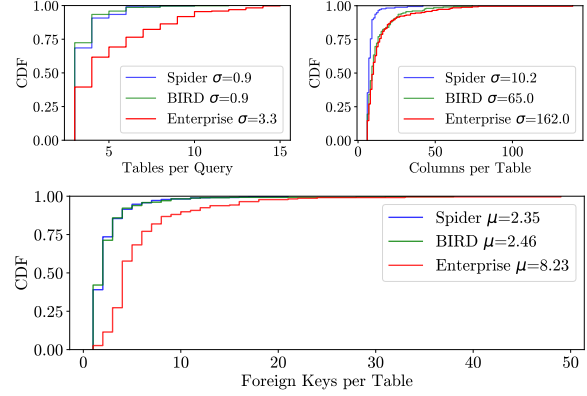


Figure 1: Cumulative Distribution Function (CDF) of an Enterprise Database Compared to Popular Benchmarks.

to as "verifiable" (Drori et al., 2025). One such amenable tasks is *Text-to-SQL*: given a natural language question, generate an SQL query that answers it (Katsogiannis-Meimarakis and Koutrika, 2023). Although *Text-to-SQL* belongs to a wider class of code generation tasks (Liu et al., 2023), it involves several unique challenges, including the selection of the relevant database tables and columns to involve in the query (*table retrieval*).

«Novak: Reviewer 3: "no proper description of the enterprise dataset": do a bit better dataset description job. Maybe a separate section.» «Rajpreet: Give simple description of nature of the database, maybe make a figure to compare spider/bird and enterprise.»

Given a natural language question q , we can define the problem of table retrieval as selecting the smallest set of table columns $\mathcal{T}$ (the *relevant set*) that contains sufficient information to answer the question. While benchmark datasets such as Spider (Yu et al., 2018) and BIRD (Li et al., 2023b) provide the relevant set as an input alongside q , this approach is impractical in real-world enterprise settings, where the relevance set of an arbitrary question is typically unknown. Therefore, it is

necessary to identify the relevant set from q and pass it to the a language model-based generator, which then outputs the corresponding SQL query.

Besides the presupposition of the relevant set, we also observe significant statistical divergence between the popular benchmarks and typical enterprise databases, primarily with respect to the complexity of questions and database schema. As depicted in *Fig. 1*, queries in Spider and BIRD are significantly less likely to require large relevant sets compared to the *Enterprise* case. While around 1/4 of *Enterprise* queries necessitate 7 or more tables in their relevant set, this probability is negligible in Spider and BIRD, while their distributions also exhibit much smaller variance (0.9 versus 3.3). In other words, the popular benchmarks evaluate only a negligible amount of queries with many joins, the ones that often incur significant performance overheads. We observe a similar effect of missing out on "large queries" at the database schema level as well. While the median number of foreign keys per table is under 2.5 in Spider and BIRD, *Enterprise* exhibits more than 3 times larger median with a non-negligible amount of tables with 10 or more foreign keys.

When it comes to the size of tables with respect to the number of columns, BIRD in fact improves on top of Spider exhibiting a similar distribution to our *Enterprise* case, except for the standard deviation that is noticeably larger in *Enterprise*. Therefore, we believe that Spider and BIRD are insufficiently representative of enterprise-scale *Text-to-SQL* tasks. Considering the above factors, we present the following contributions:

- *CSR-RAG*, a novel table retrieval system that simultaneously exploits semantic context of the question, the database schema, and its connectedness. Our two-stage process first executes contextual and graph retrieval in parallel, then combines their results using a hypergraph ranking subsystem.
- We evaluate the accuracy of *CSR-RAG* on an anonymized enterprise database achieving the precision of over 40% and recall over 80% for the most cases, satisfying the needs of modern LLM-based SQL query generators.
- Due to its parallelizable design, *CSR-RAG* achieves a superior query generation-time latency of around 30 milliseconds on commodity hardware.

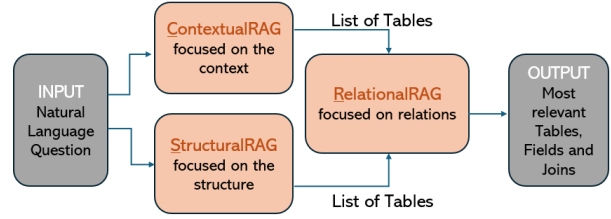


Figure 2: System Architecture.

The rest of this paper is organized as follows, in *Section 2* we present the architecture of *CSR-RAG* focusing on each individual component. In *Section 3*, we evaluate the precision, recall, and latency of *CSR-RAG*. In *Section 4*, we compare *CSR-RAG* to *Text-to-SQL* approaches from the literature and conclude our paper in *Section 5*.

2 Solution Architecture

CSR-RAG's architecture comprises of three principal components: Contextual RAG, Structural RAG, and Relational RAG, organized as depicted in *Fig. 2* with the natural language question as the input and the list of most relevant table columns as the output supplied to the downstream LLM-based query generator. Contextual RAG incorporates natural language context following the intuition that contextually similar sentences yield similar SQL queries. Structural RAG turns database schema into its knowledge graph representation supplying the retriever with the knowledge of the database structure. At the end, Relational RAG captures the outputs of the parallel execution of Contextual and Structural RAGs and models multi-entity relationships within the narrowed table scope, producing the table columns that will participate in the joins of the final SQL query.

2.1 Contextual RAG

Capturing the context of a natural language questions is crucial in measuring its similarity to the others in the ground truth data set. *<<Novak: Add a sentence to explain why the measure of similarity is needed.>>* *<<Rajpreet: Again, a fundamental step in retrieval systems. To compare the input query to whatever context we have around the prior knowledge>>* To allow for such a function, our Contextual RAG first performs sentence embedding using a BERT-like encoder (Reimers and Gurevych, 2019), mapping the sentence into a multi-dimensional vector space. By doing so, we avoid the semantic isolation

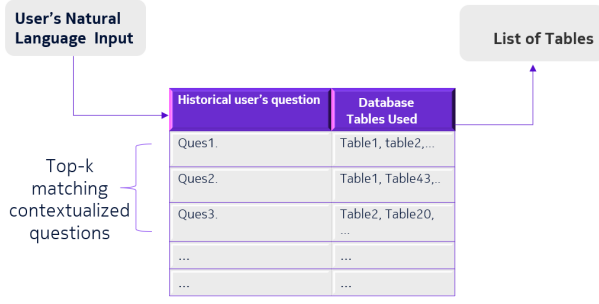


Figure 3: The Function of Contextual RAG.

drawbacks of some traditional RAGs. [«Novak: "BERT-like encoder helps avoid semantic isolation drawbacks of some traditional RAGs."»](#)

As in Fig. 3, Contextual RAG then retrieves a list of k top similar questions from the ground truth data set and associates the retrieved similar questions to their relevant sets (tables used in the query), then returns the union of all k relevant sets. [«Novak: addressing the Reviewer 3: either omit contextual RAG as a component or redefine the context not to be the ground truth set but the table descriptions or something else.»](#)

Our Contextual RAG enables global search across diverse database schemas, pinpointing domain-specific structures without requiring explicit identification. It requires two tuning parameters, the number of chunks to be retrieved and the quality of contextualization, which we need to balance to allow for comprehensive retrieval while retaining computational efficiency. Through our experiments, we observed that simpler similarity metrics such as cosine similarity yield comparable results to more complex ones such as BM25 (Sparck Jones et al., 2000).

Let Q be the set of all natural language queries, and let $\mathcal{D}$ represent the database schema consisting of T tables and F fields. Each SQL query S corresponds to a natural language question $q \in Q$, forming a chunk $C_i = (q_i, S_i)$, for $i \in \{1, 2, \dots, N\}$, where N is the count of retrieval units in the ground truth dataset. We define the contextualization function ϕ that enhances a query by incorporating schema-related information as: [«Novak: Reviewer 3: Is \$\mathcal{D}\$ the input to our RAG? No, we don't use it in retrieval time, but only when we are contextualizing the ground-truth dataset.»](#)

$$\phi(q) = q + \text{desc}(S, \mathcal{D}),$$

where $\text{desc}(S, \mathcal{D})$ includes table names, column

names, and their textual descriptions. We then use our similarity metric Sim and tuning parameter k to retrieve the similar chunks as:

$$\mathcal{R}(q') = \text{argmax}_{C_i} \text{Sim}(q', C_i), \quad |\mathcal{R}(q')| = k.$$

The larger k , the more tables retrieved and the larger the probability of retrieving unnecessary information. Therefore, we set k to optimize for precision and recall, with more details in Section 3.

2.2 Structural RAG

Effective representation of prior knowledge is paramount in retrieval systems, potentially offering substantial accuracy gains and the knowledge graphs emerged as a de-facto standard in this area. While conventional approaches use language models to generate graph triplets $\langle \text{subject}, \text{predicate}, \text{object} \rangle$, they suffer from inconsistent predicate inference. To overcome this drawback and keep the retrieval latency low, we opt for a deterministic approach to building the knowledge graph from database schema. Our graph representation consists of $\langle \text{field}, \text{"Relation"}, \text{table} \rangle$ triplets, where "Relation" denotes "is a column of" (see Fig. 4). Therefore, the total number of triplets is equal to the number of tables times the number of columns in the schema, optimizing for representation accuracy in representing enterprise-scale databases.

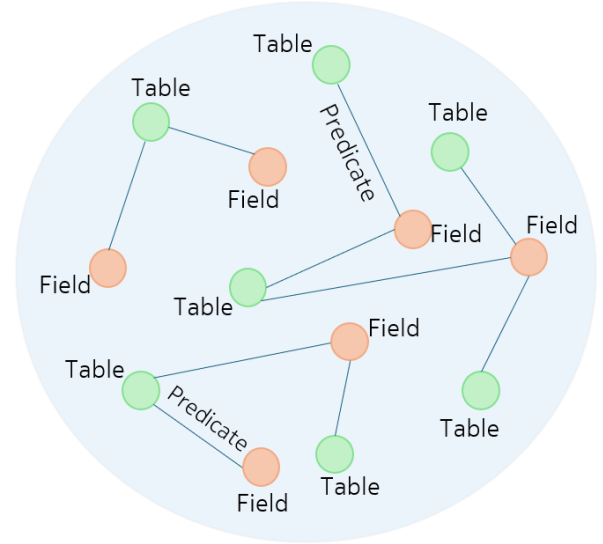


Figure 4: Graph Representation of the Schema.

Let the database schema have a set of tables T and fields F . The knowledge representation is then a graph $G = (V, E)$, where $V = T \cup F$ and $E \subseteq V \times V$. Each triplet in the knowledge graph is represented as $(f_i, t_j) \in F \times T$, where $f_i \in F$

is a column and $t_j \in T$ is a table. Given a natural language question q and a tuning parameter l , the retrieval function γ returns the top- l triplets:

$$\gamma(q, G, l) \rightarrow \{(f_i, t_j)_1, \dots, (f_i, t_j)_l\},$$

where the tuning parameter l controls the number of retrieved triplets from the knowledge graph. Similar to Contextual RAG, a large l retrieves more tables.

2.3 Relational RAG

The final component of the system addresses a critical aspect of database operations: foreign key relationships, which are essential for generating accurate SQL queries when leveraging fine-tuned LLMs. The Relational RAG component captures relationships among tables retrieved by Contextual and Structural RAGs, ensuring that the table joins are contextually relevant to the natural language query. Unlike the simpler knowledge graph representation in Structural RAG, Relational RAG encodes its knowledge into a hypergraph to handle the increased complexity, as shown in Fig. 5. We represent tables as the nodes in the hypergraph and columns as its hyperedges. Nodes are connected by a hyperedge if the column encoded by the hyperedge is likely to be joined on in an SQL query. The columns that are likely to join several tables are encoded as the hyperedges with many nodes. This modeling principle is particularly suitable for database schemas with complex foreign-key relationships such as our *Enterprise* example from Section 1, as it allows for flexible modeling of multi-table dependencies. The function of Re-

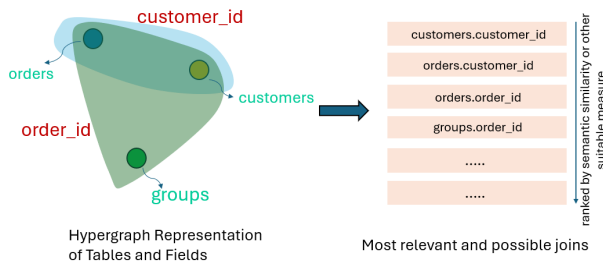


Figure 5: Hypergraph Ranking to Select the Most Relevant Scoped Columns.

lational RAG therefore consists of: 1) dynamic hypergraph construction from narrowed table set outputted by Contextual and Structural RAGs, 2) capturing the semantics of SQL joins, and 3) preparing the table.column format for the fine-tuned LLM query generator.

Algorithm 1 Hypergraph Ranking.

```

1:  $h$  : Number of nodes to select
2:  $N$  : Set of nodes
3:  $T$  : Task metadata requirements
4:  $O$  : Operation functions for scoring
5:  $W$  : Weight factors for different metrics
6: func HypergraphRanking ( $h, N, T, O, W$ )
7:  $scoredNodes \leftarrow \emptyset$ 
8: for all node  $v \in N$  & hyperedge  $e \in T$ 
9:    $M_v \leftarrow$  metadata of node  $v$ 
10:  if  $M_v$ .availability = false then
11:    continue
12:  end if
13:   $(v \otimes e) \leftarrow$  compute with operator;  $\otimes \in O$ 
14:  if  $w_v > 0$  where  $w_v \in W$  then
15:     $score_{v,e} \leftarrow (v \otimes e)/w_v$ 
16:  else
17:     $score_{v,e} \leftarrow 0$ 
18:  end if
19:   $scoredNodes \leftarrow scoredNodes \cup \{(v, score_{v,e})\}$ 
20: end for
21: SORT  $scoredNodes$  by score in descending order
22:  $selectedNodes \leftarrow$  First  $h$  nodes from  $scoredNodes$ 
23: RETURN  $selectedNodes$ 

```

The core of Relational RAG is our general operator-based hypergraph ranking algorithm (see Algorithm 1), which relies on ranking hyperedge-node concatenations to identify potential joins. Similarly to the previous RAG components, Relational RAG makes use of a similarity metric to pick the top- k hyperedge-node pairs, effectively prioritizing the most relevant valid table joins based on the context of the input questions. By integrating contextual and structural information from previous components and leveraging the hypergraph ranking, Relational RAG ensures sufficiently accurate yet efficient retrieval of table-join information, enhancing SQL generation for complex queries (see Fig. 5).

Let $H = (V, E)$ be a hypergraph where V is the set of tables retrieved from Contextual and Structural RAGs, E are the hyperedge representations of columns, and $\Upsilon(v, e)$ is the relevance score of the semantic entity $(v \otimes e)$ in the context of the natural language question q , where $\otimes$ denotes concatenation of a table and its column. The result of Relational RAG is then output of the similarity

Group	Tables	Columns
1	50	701
2	100	1486
3	200	2567
4	246	3021

Table 1: The Sizes of Experimental Table Groups in the *Enterprise* dataset.

ranking:

$$\beta(q, H) \rightarrow \{(v_1 \otimes e_1), (v_2 \otimes e_2), \dots, (v_k \otimes e_h)\},$$

where (e_i, v_i) are ranked and ordered from the entire hypergraph representation. The ranking function β performs a similarity measure between the question q and the hypergraph H using $\Upsilon(v, e)$, which we implement as cosine similarity of the semantic entity $(v \otimes e)$ and question q . The tuning parameter h defines the number of semantic entities to be retrieved by the Relational RAG component. Note that the hypergraph ranking algorithm can take various forms depending on the semantic meaning attached to $\otimes$. For instance, one can add more context around this operator to involve tables and column descriptions, which would further enhance the retrieval accuracy.

3 Experimental Evaluation

In this section, we evaluate *CSR-RAG* on our anonymized *Enterprise* database from Section 1. We consider four groups of database schemas and their related ground truth traces consisting of natural language questions and corresponding SQL queries. The number of tables and columns per group are given in Table 1. In our evaluations, we focus on iterative implementations of Contextual, Structural, and Relational RAGs, where iterations represent recurrent passes through the RAG component. For instance, the second iteration of Contextual RAG takes the output of the first iteration of Contextual RAG alongside a decreased tuning parameter. If not otherwise noted, we write the hyperparameter values in each iteration as the numbers above the data point in our plots.

«Novak: Reviewer 3: better job at describing hyper parameters.» «Rajpreet: Methodology to come at the right combination of hyper-parameters.»

In Fig. 6, we plot the precision and recall of Contextual RAG as a function of the number of

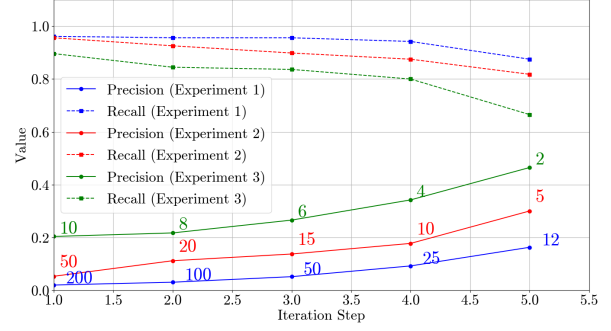


Figure 6: Precision and Recall for Contextual RAG on Group 2.

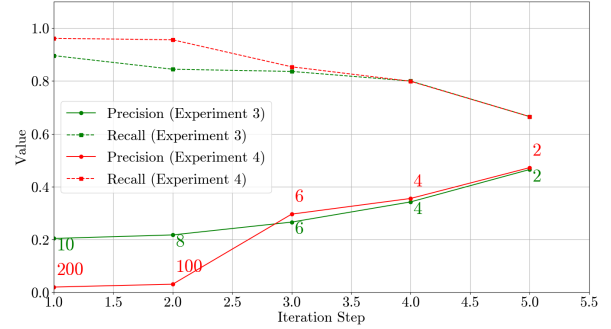


Figure 7: Precision and Recall for Contextual RAG on Group 2.

iterations. As apparent from the bond of solid lines in Fig. 6, a decreasing chunk size k leads to an increased Contextual RAG precision. This trend is expected due to a reduction in the search space across the iterations, defining the search space as the space of all possible tables in the experimental table group. However, we also see a moderate decrease in recall attributed to an increased number of false negatives, which is also the consequence of the reduced number of selected tables.

As shown in Fig. 7, the initial choice of k only moderately affects precision. «Novak: Clarify for Reviewer 3. The INITIAL choice of k does not matter.» The choice of a very small k in a single iteration does not lead to a high precision hike due to the insufficient contextual information being retrieved, resulting in potentially incomplete or inaccurate answers. The loss of relevant contextual information is high in a single step retrieval with small k , for instance 2 or 4.

Similarly to Contextual RAG, the precision and recall for Structural RAG are shown in Fig. 8. The trend is similar to that of Contextual RAG, but there is a sharp drop in recall due to lack of contextual information in the knowledge graph representation

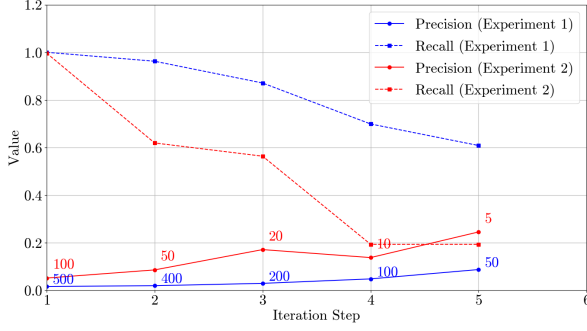


Figure 8: Precision and Recall for Structural RAG on Group 2.

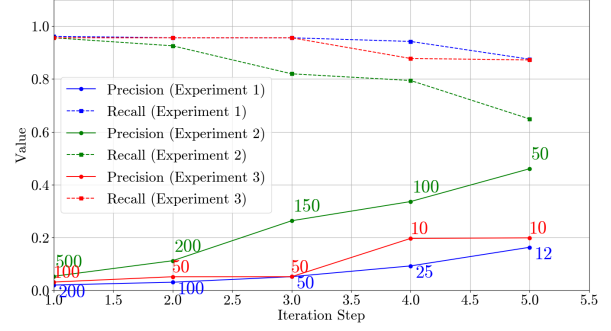


Figure 10: Precision and Recall for the Relational RAG Component on Group 3.

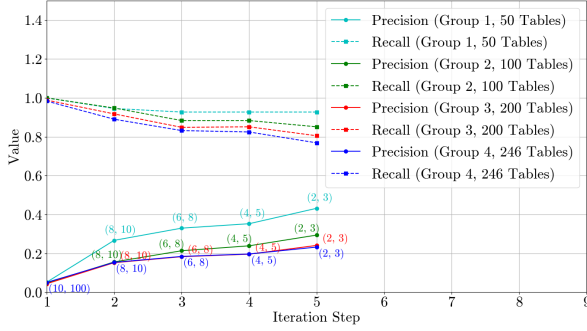


Figure 9: Precision and Recall for Contextual and Structural RAG Components Combined for all Groups.

of the database schema. The tuning parameter l , the size of retrieved triplets from the knowledge graph are highlighted at the data points in Fig. 8.

«Novak: Reviewer 3: better discussion on how to choose the optimal k balancing precision and recall.» «Rajpreet: Optimal balance is highly subjective to the quality of prior knowledge. And also how complex the knowledge graph.»

A combined evaluation of Contextual and Structural RAG is shown in Fig. 9. For each group, the best configuration of tuning parameters (k, l) is highlighted at each iteration step. The choice of these parameters might differ among ground truth data sets, but we do not expect significant changes in the trend. After the retrieval of tables using Contextual, and Structural RAG, our system performs ranking using hypergraph representation of selected tables to retrieve most relevant columns that are necessary for generating efficient joins. Following the same conventions from the previous plots, we plot the precision and recall results of Relational RAG in Fig. 10.

Finally, we measure the incurred latency of entire CSR-RAG on all groups and plot the results

in Fig. 11 reporting on the 50th, 90th, and 99th percentile. Our results suggest that CSR-RAG takes around 30ms on average to perform the full table retrieval task, which is negligible compared to the time the state-of-the-art LLM-based query generators take to generate the SQL queries. We obtain the latency results from Fig. 11 on Intel Xeon Gold 5317 with 3GHz cores, a relatively common data-center CPU.

4 Related Work

(Huang et al., 2022) propose an optimized dense retriever for joint table-text evidences for OpenQA. (Chen et al., 2025) highlight the "table-retrieval" problem in Text-to-SQL and propose a re-ranking method based on mixed integer programming to select the best set of tables based on joint decisions about relevance and compatibility. As opposed to these works, we not only retrieve tables, but also the columns involved in potential joins, thus optimizing for complex large-scale databases. We also do it in a computationally tractable and deterministic fashion as opposed to non-deterministic retrieval systems such as (Li et al., 2023a), (Mao et al., 2024), (Li et al., 2024), (Wang et al., 2023), while our knowledge graph-based schema representation enables for more accurate question answering as suggested by (Sequeda et al., 2024). On the other hand, a recent work on hybrid retrieval system called HYBGRAG (Lee et al., 2024) takes an "agentic" approach and therefore suffers from prohibitive latency overheads. Another work based on a classification based table selection (Chopra and Azam, 2024) offers a practical solution for improving the accuracy and efficiency of Text-to-SQL (Sun et al., 2023), (Jiao et al., 2024), (Xiang et al., 2023) models, but requires a huge training dataset and

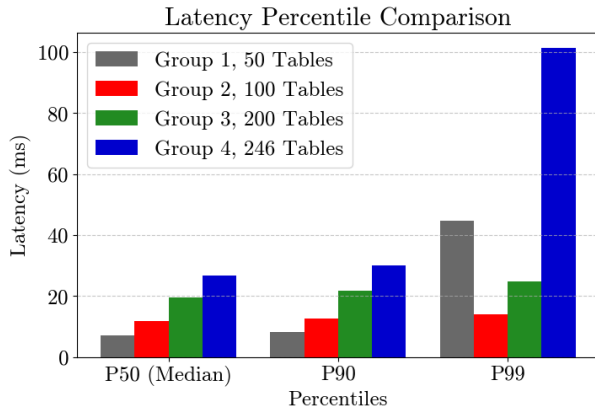


Figure 11: Overall Latency of *CSR-RAG*.

struggles to scale for enterprise databases. Noteworthy is (Hu et al., 2023) that offers a synthesis framework to generate high-quality data for *Text-to-SQL* due to the lack of public datasets from enterprise databases.

5 Conclusions

In this work, we presented *CSR-RAG*, a system that addresses a critical challenge in information retrieval for *Text-to-SQL*. In *CSR-RAG*, we divide the retrieval problem into three inherent aspects: the semantic context of natural language questions, database structure, and connectedness among the database tables. We achieve a sufficiently high precision and recall for enterprise-scale databases combining the output from Contextual and Structural RAGs within our hypergraph ranking-based Relational RAG. Due to *CSR-RAG*’s superior latency over similar approaches from the literature, we use *CSR-RAG* as a real-time inferencing component in front of a state-of-the-art LLM-based SQL query generator, achieving industry-grade SQL generation accuracy and an exceptional run-time performance.

References

Peter Baile Chen, Yi Zhang, and Dan Roth. 2025. *Is table retrieval a solved problem? exploring join-aware multi-table retrieval*. *Preprint*, arXiv:2404.09889.

Ankush Chopra and Rauful Azam. 2024. *Enhancing natural language query to sql query generation through classification-based table selection*. In *International Conference on Engineering Applications of Neural Networks*.

Iddo Drori, Gaston Longhitano, Mao Mao, Seunghwan Hyun, Yuke Zhang, Sungjun Park, Zachary Meeks,

Xin-Yu Zhang, Ben Segev, Howard Yong, Nakul Verma, Avi Shporer, Alon Amit, and Madeleine Udell. 2025. *Diverse inference and verification for advanced reasoning*. *Preprint*, arXiv:2502.09955.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Cheng-gang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. *Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning*. *Preprint*, arXiv:2501.12948.

Yiqun Hu, Yiyun Zhao, Jiarong Jiang, Wuwei Lan, Henghui Zhu, Anuj Chauhan, Alexander Hanbo Li, Lin Pan, Jun Wang, Chung-Wei Hang, Sheng Zhang, Jiang Guo, Mingwen Dong, Joseph Lilien, Patrick Ng, Zhiguo Wang, Vittorio Castelli, and Bing Xiang.

511	2023. Importance of synthesizing high-quality data for text-to-SQL parsing . In <i>Findings of the Association for Computational Linguistics: ACL 2023</i> , pages 1327–1343, Toronto, Canada. Association for Computational Linguistics.	
512		
513		
514		
515		
516	Junjie Huang, Wanjun Zhong, Qian Liu, Ming Gong, Daxin Jiang, and Nan Duan. 2022. Mixed-modality representation learning and pre-training for joint table-and-text retrieval in openqa . <i>Preprint</i> , arXiv:2210.05197.	
517		
518		
519		
520		
521	Yizhu Jiao, Sha Li, Sizhe Zhou, Heng Ji, and Jiawei Han. 2024. Text2DB: Integration-aware information extraction with large language model agents . In <i>Findings of the Association for Computational Linguistics: ACL 2024</i> , pages 185–205, Bangkok, Thailand. Association for Computational Linguistics.	
522		
523		
524		
525		
526		
527	George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A survey on deep learning approaches for text-to-sql . <i>The VLDB Journal</i> , 32(4):905–936.	
528		
529		
530	Meng-Chieh Lee, Qi Zhu, Costas Mavromatis, Zhen Han, Soji Adeshina, Vassilis N. Ioannidis, Huzefa Rangwala, and Christos Faloutsos. 2024. Hyb-grag: Hybrid retrieval-augmented generation on textual and relational knowledge bases . <i>Preprint</i> , arXiv:2412.16311.	
531		
532		
533		
534		
535		
536	Chunhui Li, Yifan Wang, Zhen Wu, Zhen Yu, Fei Zhao, Shujian Huang, and Xinyu Dai. 2024. MultiSQL: A schema-integrated context-dependent Text2SQL dataset with diverse SQL operations . In <i>Findings of the Association for Computational Linguistics: ACL 2024</i> , pages 13857–13867, Bangkok, Thailand. Association for Computational Linguistics.	
537		
538		
539		
540		
541		
542		
543	Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023a. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls . <i>Preprint</i> , arXiv:2305.03111.	
544		
545		
546		
547		
548		
549		
550		
551	Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Ma Chenhao, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023b. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls . In <i>Advances in Neural Information Processing Systems</i> , volume 36, pages 42330–42357. Curran Associates, Inc.	
552		
553		
554		
555		
556		
557		
558		
559		
560	Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation . In <i>Proceedings of the 37th International Conference on Neural Information Processing Systems</i> , NIPS ’23, Red Hook, NY, USA. Curran Associates Inc.	
561		
562		
563		
564		
565		
566		
	Wenxin Mao, Ruiqi Wang, Jiyu Guo, Jichuan Zeng, Cuiyun Gao, Peiyi Han, and Chuanyi Liu. 2024. Enhancing text-to-SQL parsing through question rewriting and execution-guided refinement . In <i>Findings of the Association for Computational Linguistics: ACL 2024</i> , pages 2009–2024, Bangkok, Thailand. Association for Computational Linguistics.	567
		568
		569
		570
		571
		572
		573
	Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training .	574
		575
		576
	Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks . <i>CoRR</i> , abs/1908.10084.	577
		578
		579
	Juan Sequeda, Dean Allemang, and Bryon Jacob. 2024. A benchmark to understand the role of knowledge graphs on large language model’s accuracy for question answering on enterprise sql databases . In <i>Proceedings of the 7th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA)</i> , GRADES-NDA ’24, New York, NY, USA. Association for Computing Machinery.	580
		581
		582
		583
		584
		585
		586
		587
		588
	K. Sparck Jones, S. Walker, and S.E. Robertson. 2000. A probabilistic model of information retrieval: development and comparative experiments: Part 1 . <i>Information Processing & Management</i> , 36(6):779–808.	589
		590
		591
		592
	Shuo Sun, Yuze Gao, Yuchen Zhang, Jian Su, Bin Chen, Yingzhan Lin, and Shuqi Sun. 2023. An exploratory study on model compression for text-to-SQL . In <i>Findings of the Association for Computational Linguistics: ACL 2023</i> , pages 11647–11654, Toronto, Canada. Association for Computational Linguistics.	593
		594
		595
		596
		597
		598
	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need . In <i>Advances in Neural Information Processing Systems</i> , volume 30. Curran Associates, Inc.	599
		600
		601
		602
		603
	Bing Wang, Yan Gao, Zhoujun Li, and Jian-Guang Lou. 2023. Know what I don’t know: Handling ambiguous and unknown questions for text-to-SQL . In <i>Findings of the Association for Computational Linguistics: ACL 2023</i> , pages 5701–5714, Toronto, Canada. Association for Computational Linguistics.	604
		605
		606
		607
		608
		609
	Yanzheng Xiang, Qian-Wen Zhang, Xu Zhang, Ziejie Liu, Yunbo Cao, and Deyu Zhou. 2023. g³r: A graph-guided generate-and-rerank framework for complex and cross-domain text-to-SQL generation . In <i>Findings of the Association for Computational Linguistics: ACL 2023</i> , pages 338–352, Toronto, Canada. Association for Computational Linguistics.	610
		611
		612
		613
		614
		615
		616
	Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task . In <i>Proceedings of the 2018</i>	617
		618
		619
		620
		621
		622

623 *Conference on Empirical Methods in Natural Lan-*
624 *guage Processing*, pages 3911–3921, Brussels, Bel-
625 *gium*. Association for Computational Linguistics.